is in *int* form, is divided by the value of 'z', i.e., 8. This yields the result 75, which is in *int* form. Since 75 falls within the range (-128 to +127), without any side effect, this value is assigned to the 'res' variable after demoting it to *signed char*.

Table 2 gives the explanation for the disassembled code shown in Figure 9.

Table 2

Assembly instruction	Description
movsbl 0x1c(%esp),%edx	Sign-extending a single byte (the value of x, which is 20) pointing by *esp* and copying it into a double-word destination *edx*
movsbl 0x1d(%esp),%eax	Sign-extending a single byte (the value of y, which is 30) pointing by *esp* and copying it into a double-word destination *eax*
imul %edx,%eax	Multiplying two values
movsbl 0x1e(%esp),%ecx	Sign-extending a single byte (the value of *z*, which is 8) pointing by *esp* and copying it into a double-word destination *%ecx*.
idiv %ecx	Dividing the result of *x * y* by *z*
mov %al,0x1f(%esp)	Moving the result to the location pointing by *esp*

Let us consider the last example (Code-4) to understand integer promotion.

Think about whether we can apply, bitwise, the left shift operator on a *char* data type more than the bits present in the *char*? For instance, ch << 8, since the maximum left shifting possible is only seven times.

```
1 #include <stdio.h>
2 int main()
3 {
4     char ch = 0xAB;
5     char res = ch << 8;
6     printf("Res: %X\n", res);
7     return 0;
8 }
```

Code-4: Code to show how integer promotion takes place in a bitwise left-shift operation

The left-shift operator can be applied to the character variable more than the limited size, which is 7, but it is of no use, even though the compiler will not generate any errors.

```
int main()
{
 804841d:       55                              push   %ebp
 804841e:       89 e5                           mov    %esp,%ebp
 8048420:       83 e4 f0                        and    $0xfffffff0,%esp
 8048423:       83 ec 20                        sub    $0x20,%esp
        char ch = 0xAB;
 8048426:       c6 44 24 1e ab                  movb   $0xab,0x1e(%esp)
        char res = ch << 8;
 804842b:       0f be 44 24 1e                  movsbl 0x1e(%esp),%eax
 8048430:       c1 e0 08                        shl    $0x8,%eax
 8048433:       88 44 24 1f                     mov    %al,0x1f(%esp)
        printf("Res: %X\n", res);
 8048437:       0f be 44 24 1f                  movsbl 0x1f(%esp),%eax
 804843c:       89 44 24 04                     mov    %eax,0x4(%esp)
 8048440:       c7 04 24 f0 84 04 08            movl   $0x80484f0,(%esp)
 8048447:       e8 a4 fe ff ff                  call   80482f0 <printf@plt>
        return 0;
 804844c:       b8 00 00 00 00                  mov    $0x0,%eax
```

Figure 10: Disassembled code for the source Code-4

In this case, the output will always be zero. An in-depth explanation can be found about disassembled code in Figure 10 and Table 3.

Table 3

Assembly instruction	Description
movb $0xab,0x1e(%esp)	Move a byte(0xAB) of info into the location pointing by *esp*
movsbl 0x1e(%esp),%eax	Sign-extending a single byte (the value of *ch*, which is 0xAB) pointing by *esp* and copying it into a double-word destination *eax*
shl $0x8,%eax	Left-shift eight times on integer value
mov %al,0x1f(%esp)	Move the result which is in *al* register into the location pointing by *esp*

In order to improve the performance of the operations, the compiler will internally apply the implicit type conversion rules to the data types that have a lower rank than that of the 'int' data type. But the programmer has to take care of 'overflow' issues during the operations. Performing the operations on the natural size of the machine is very fast compared to the other data types whose size is not equal to the natural size of the machine, like *char* and *short*. While writing the expressions involving multiple data types, the programmer has to ensure that the expression is free of side effects. END

By: Satyanarayana Sampangi

The author is a member of the embedded software team at Emertxe Information Technology (P) Ltd (*http://www.emertxe. com*). His areas of interest are embedded C programming combined with data structures, microcontrollers and Linux Internals. He can be reached at *satya@emertxe.com, satya.2891@gmail.com.*